Appunti teorici di
Circuiti ed algoritmi per il Digital Signal Processing

Stefano Rossini

12 ottobre 2025

Introduzione

Appunti ordinati, con approfondimenti passo-passo, del corso di Circuiti ed algoritmi per il Digital Signal Processing per il corso di laurea in Ingegneria Elettronica presso l'Università Politecnica delle Marche per l'A.A. 2025/2026.

Per quelli che seguono il nuovo corso di Ingegneria elettronica:
https://www.univpm.it/Entra/Offerta_formativa_1/Corso_di_laurea_triennale_in_Ingegneria_Elettronica_e_delle_Tecnologie_Digitali
cioè la Laurea in Ingegneria Elettronica e delle Tecnologie Digitali, il corso prende il nome di Digital Signal Processing.

Alla fine della scrittura di ogni capitolo, ricordati di correggere l'ortografia usando Texstudio e non VS Code.

Inoltre, controllare i riferimenti in fondo ad ogni capitolo siano corretti.

Indice

1 Segnali e circuiti a tempo discreto **5**
1.1 Sequenze . . . 6
1.1.0.1 Sequenza impulso unitario . . . 8
1.1.0.2 Sequenza impulso unitario ritardato . . . 9
1.1.1 Proprietà di campionamento . . . 10
1.1.2 Sequenza elementari . . . 11
1.1.2.1 Gradino unitario . . . 11
1.1.2.2 Rampa unitaria . . . 13
1.1.2.3 Sequenze esponenziali reali . . . 14
1.1.2.4 Sequenze esponenziali complesse . . . 15
1.1.3 Sequenze periodiche . . . 16
1.1.3.1 Proprietà dell'esponenziale complesso puro . . . 17
1.2 Sistemi discreti . . . 18
1.2.1 Sistemi lineari e stazionari . . . 19
1.2.1.1 Sistema discreto lineare . . . 20
1.2.1.2 Sistema discreto stazionario . . . 21
1.2.2 Sistemi causali e stabili . . . 22
1.2.3 Sistemi discreti causali . . . 23
1.2.4 Sistemi discreti stabili BIBO . . . 24
1.3 Risposta impulsiva in un sistema LTI . . . 25
1.3.1 Prodotto di convoluzione e le sue proprietà . . . 27
1.3.2 Come calcolare la risposta impulsiva in un sistema LTI . . . 28
1.3.3 Risposta impulsiva in un sistema causale e stabile BIBO . . . 29
1.4 Suddivisione dei sistemi LTI . . . 31
1.4.1 Esempio numerico di stabilità di un sistema IIR . . . 33
1.4.2 Analisi di esempi di sistemi LTI . . . 34
1.5 Connessioni dei sistemi LTI . . . 36
1.6 Sistemi LTI descritti da equazioni alle differenze . . . 38

2 Introduzione al MATLAB **41**
2.1 Matlab ed i suoi utilizzi . . . 42
2.2 Il sistema Matlab, l'IDE ed i Data Types . . . 44
2.3 Operazioni su matrici e vettori . . . 46
2.4 Plotting . . . 51
2.5 Esercizi di esempio . . . 53
2.6 Matlab Programming . . . 54
2.6.1 Script file . . . 55
2.6.2 Funzioni . . . 56
2.7 Convoluzione di due sequenze . . . 57

Domande e note varie

2025-10-12 Ora 18:10

TO-DO Cosa fare per la prossima volta

- Continuare dagli appunti di 2025-10-08 per il capitolo 3 - DTFT

TO-DO Domande per il prof per il 2025-10-14

- 2.7 Convoluzione di due sequenze pag 57
- 2.7 Convoluzione di due sequenze pag 58
- 2.7 Convoluzione di due sequenze pag 58
- 2.7 Convoluzione di due sequenze pag 58

Questa è una box rossa di esempio

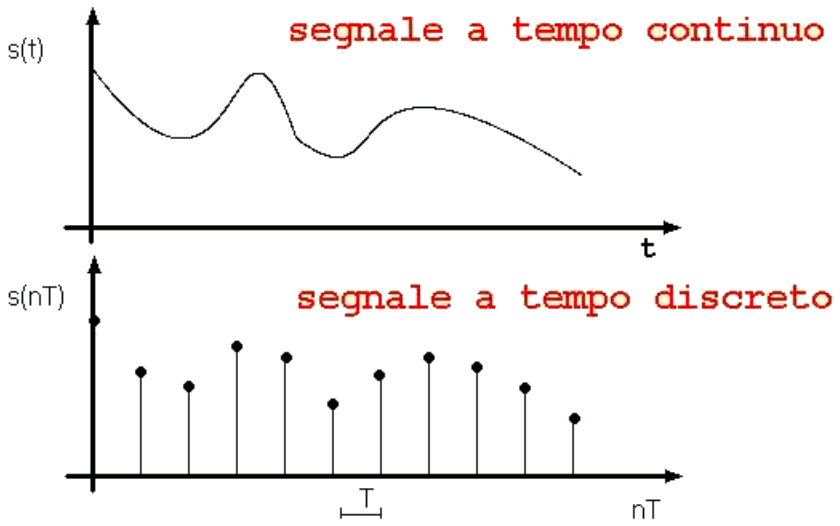
Questa è una box verde di esempio

TO-DO per il file in generale

- Fare un Latex a parte con la spiegazione di tutti i codici passo-passo (o un Latex)
- Fare più disegni di quello che si sta spiegando

Capitolo 1

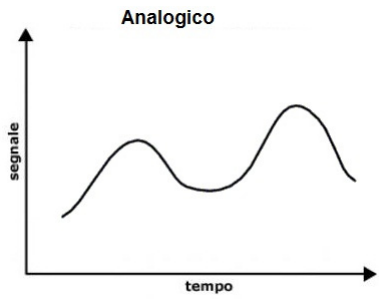
Segnali e circuiti a tempo discreto



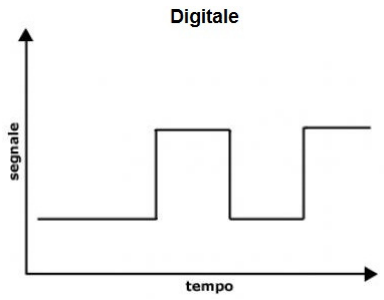
1.1 Sequenze

[1]

I segnali analogici sono dei segnali che hanno valori a tempo continuo ed a valori continui:



Invece, nei circuiti digitali, avremo a che fare con dei segnali discreti sia nel tempo che nei valori, come il seguente:

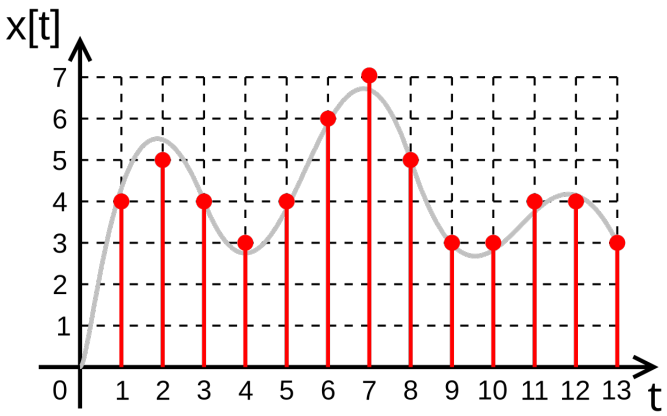


La figura qui presente è sbagliata perchè è un segnale a tempo continuo ed a valori discreti, ma è utilizzata solo per distiguere la differenza tra segnale analogico e discreto.

Nella forma più generale, non si considera il segnale come segnale discreto, bensì come sequenza, perchè, se si intende una sequenza come un segnale, significa che associamo quella sequenza a un segnale fisico.

È una sottigliezza quella ultima elencata, ma è quel particolare che permette di dividere il "mondo fisico" dal "mondo matematico", cioè quello del DSP.

Un esempio di sequenza:



[1] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 2

Dal punto di vista grafico, si considera l'interpolazione del punto (cioè quella linea che collega il t-esimo punto al x[t-esimo] punto).

Nella realtà, se si considera una sequenza, si intende SOLO il punto senza interpolazione.

Quindi, come ha detto il prof a lezione, per sequenza si intende un insieme ordinato di punti.

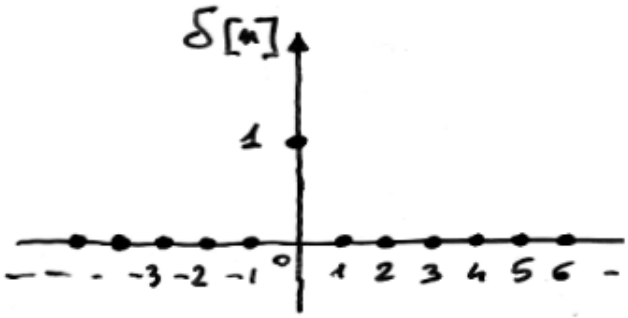
A livello di notazione, per distinguere i segnali analogici dalle sequenze discrete si utilizzeranno le parentesi quadre [ ] piuttosto che le parentesi tonde ( ).

1.1.0.1 Sequenza impulso unitario

[2]

La sequenza elementare alla base di tutte le sequenze è la sequenza impulso unitario.

Di seguito dal punto di vista grafico la sequenza impulso unitario:



La sequenza $\delta[n]$ impulso unitario gode della sequenza proprietà:

$$\delta[n] = \begin{cases} 0 \text{ per } n \neq 0 \\ 1 \text{ per } n = 0 \end{cases} \tag{1.1}$$

cioè, scritta a parole, vale 1 per n e vale 0 per tutti gli altri valori di n.

Se avete svolto e/o seguito l'esame di Teoria dei Segnali (o Segnali Determinati e Aleatori) con Franco Chiaraluce, la sequenza impulso unitario $\delta[n]$ NON è la Delta di Dirac $\delta(t)$, dove:

$$\delta(t) = \begin{cases} 0 \text{ per } t \neq 0 \\ +\infty \text{ per } t_0 = 0 \end{cases} \tag{1.2}$$

di cui gode della sequente proprietà:

$$\int_{-\infty}^{+\infty} \delta(t)dt = 1 \tag{1.3}$$

Nel caso dell'impulso unitario $\delta[n]$, n è l'indice della sequenza dove $\delta[n] = 1$.

Questo tipo di sequenza non pone problemi di definizione e realizzabilità.

[2] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 2

1.1.0.2 Sequenza impulso unitario ritardato

[3]

Un altro tipo di sequenza, che proviene dalla sequenza impulso unitario, è la sequenza impulso unitario ritardato $\delta[n - n_0]$.

Dal punto di vista matematico, la sequenza impulso unitario ritardato è la seguente:

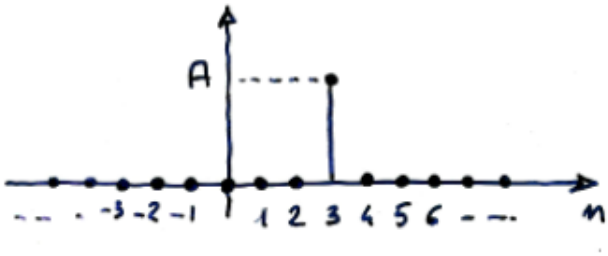
$$\delta[n - n_0] = \begin{cases} 0 \text{ per } n \neq n_0 \\ 1 \text{ per } n = n_0 \end{cases} \tag{1.4}$$

cioè, è la sequenza impulso unitario $\delta[n]$ ritardato all'indice n_0.

Se moltiplichiamo la sequenza impulso unitario $\delta[n]$ per un coefficiente A e lo ritardiamo all'indice n_0:

$$n_0 = 3 \tag{1.5}$$

allora otteniamo la seguente sequenza impulso ritardato:



Dal punto di vista matematico:

$$\delta[n - 3] = \begin{cases} 0 \text{ per } n \neq 3 \\ A \text{ per } n = 3 \end{cases} \tag{1.6}$$

Quest'ultima formula ci dice che, se la sequenza impulso unitario viene moltiplicata per un coefficiente moltiplicativo A, avremo una sequenza impulso; altrimenti la sequenza impulso unitario rimane tale se $A = 1$.

Il ritardo può essere definito anche in inglese come shift o delay.

Lo shift è una proprietà di traslazione di sequenze.

[3] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 2
Appunti — 2025-09-24 — pag 3
Latex — 2025-09-30 Mattino — pag 9

1.1.1 Proprietà di campionamento

[4]

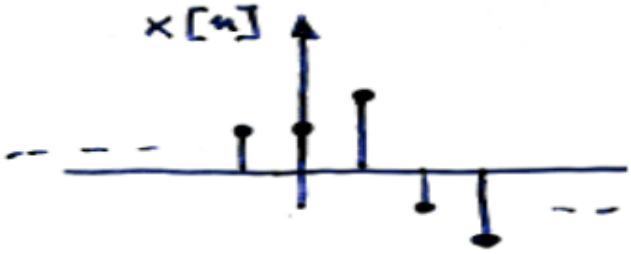
Di seguito la proprietà di campionamento.

Una qualsiasi sequenza $x[n]$ può essere espressa come somma di impulsi unitari opportunamente ritardati e amplificati.

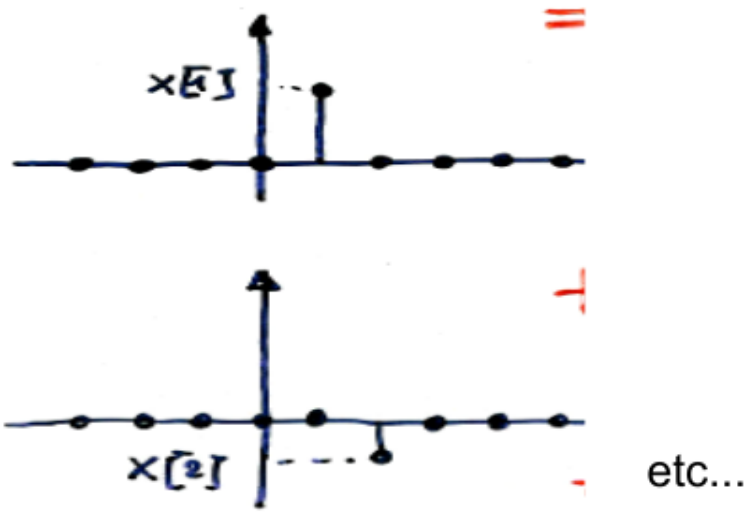
Da un punto di vista matematico, possiamo scrivere la sequenza $x[n]$ come:

$$x[n] = \sum_{k=-\infty}^{+\infty} x[k] \cdot \delta[n-k] \tag{1.7}$$

Applicando la proprietà di campionamento, possiamo visualizzare la sequenza $x[n]$:



nelle sue componenti cioè come somma dei seguenti componenti:



Rispetto al corso di Teoria dei Segnali, in questo caso si indica la proprietà di campionamento, che non è il teorema del campionamento: sono due cose differenti.

Il teorema del campionamento afferma una rappresentazione, quindi una funzione biunivoca (cioè un segnale può essere visto sia nel dominio del tempo che in quello delle frequenze).

Invece, con la proprietà del campionamento, non si cambia dominio: si rimane sempre nello stesso dominio della sequenza originale (che in questo caso è quello degli indici discreti n).

[4] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 3

1.1.2 Sequenza elementari

Potremmo elencare tutti i tipi di sequenze, ma elencheremo solo quelle elementari.

1.1.2.1 Gradino unitario

[5]

La sequenza $u[n]$ gradino unitario ha la sequenza espressione:

$$u[n] = \begin{cases} 0 \text{ per } n < 0 \\ 1 \text{ per } n \geq 0 \end{cases} \tag{1.8}$$

Da un punto di vista grafico, possiamo visualizzare il gradino unitario $u[n]$ come:

Utilizzando la proprietà dell'impulso, possiamo scrivere il gradino unitario $u[n]$ come:

$$\begin{aligned} u[n] &= \sum_{k=-\infty}^{+\infty} u[k] \cdot \delta[n-k] \\ &= \sum_{k=-\infty}^{+\infty} \delta[n-k] \\ &= \sum_{k=0}^{+\infty} \delta[n-k] \end{aligned} \tag{1.9}$$

Quindi, la sommatoria non inizia più da $k = -\infty$, bensì da $k = 0$.

Si può applicare le seguenti semplificazione perchè:

$$u[k] = 1 \ \forall k \geq 0 \tag{1.10}$$

Nel campo dei discreti, non esiste l'operatore matematico dell'integrale (che calcola l'area sotto la funzione): bensì si utilizzerà la somma.

[5] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 4

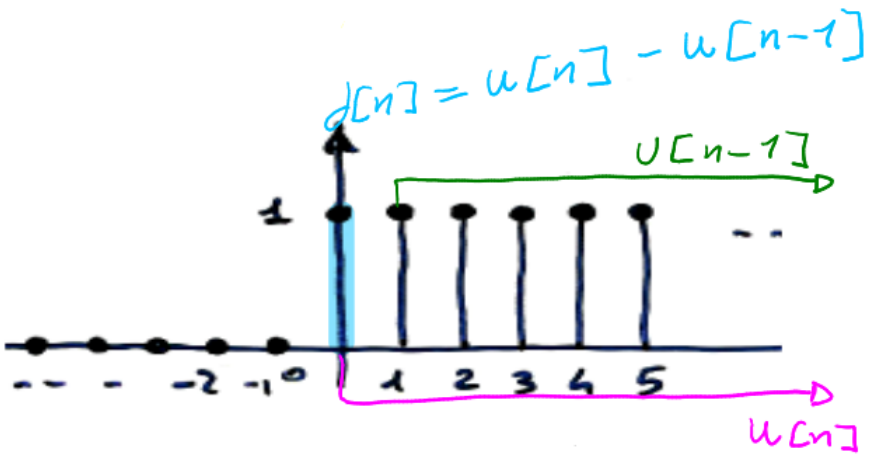
Si può verificare che:

$$\begin{cases} u[n] = \sum_{k=0}^{n} \delta[k] \\ \delta[n] = u[n] - u[n-1] \end{cases} \tag{1.11}$$

dove $\delta[n]$ è l'impulso unitario.

> Nel campo dei discreti, non esiste l'operatore matematico della derivata (che quindi calcola la differenza infinitesimale tra due punti di una funzione): bensì si utilizzerà la differenza tra una sequenza e la stessa sequenza shiftata / ritardata di un punto.

Da un punto di vista grafico, se ci poniamo nell'indice $n = 0$, l'equazione precedente possiamo visualizzarla graficamente:



Più in generale, è possibile definire una famiglia di sequenze mediante operazioni di sommatoria e differenza finita, analogamente al caso continuo.

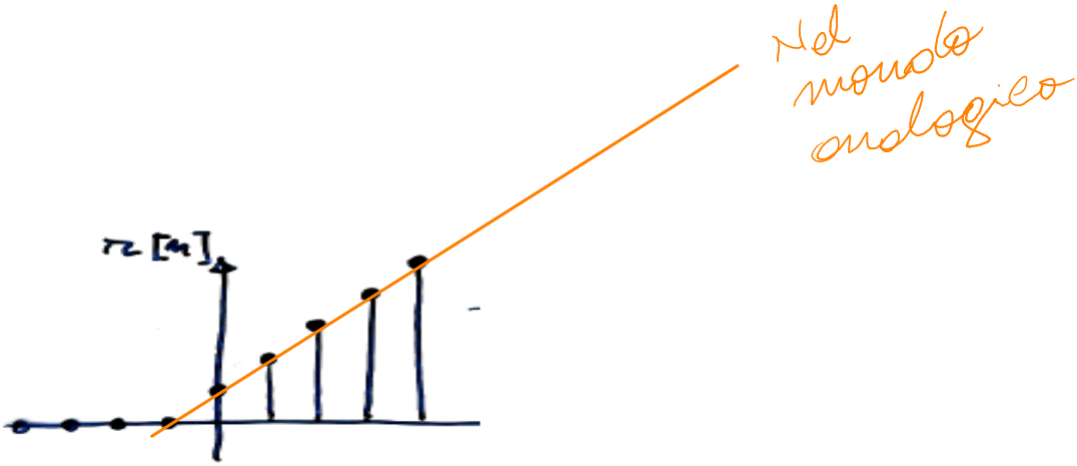
1.1.2.2 Rampa unitaria

[6]

Possiamo definire la rampa unitaria $r[n]$ come:

$$\begin{cases} r[n] = \sum_{k=-\infty}^{n} u[k] \\ u[n] = r[n] - r[n-1] \end{cases} \tag{1.12}$$

Confrontando la rampa unitaria $r[n]$ con il mondo analogico, avremo la seguente figura:



La rampa unitaria $r[n]$ viene definita unitaria perchè, per qualsiasi valore n scelto, la differenza (cioè la "derivata" tra un indice ed un altro) è sempre unitaria.

In formule:

$$u[n] = 1 \ \forall n \tag{1.13}$$

[6] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 5
Appunti — 2025-09-24 — pag 2
Latex — 2025-09-30 Mattino — pag 13

1.1.2.3 Sequenze esponenziali reali

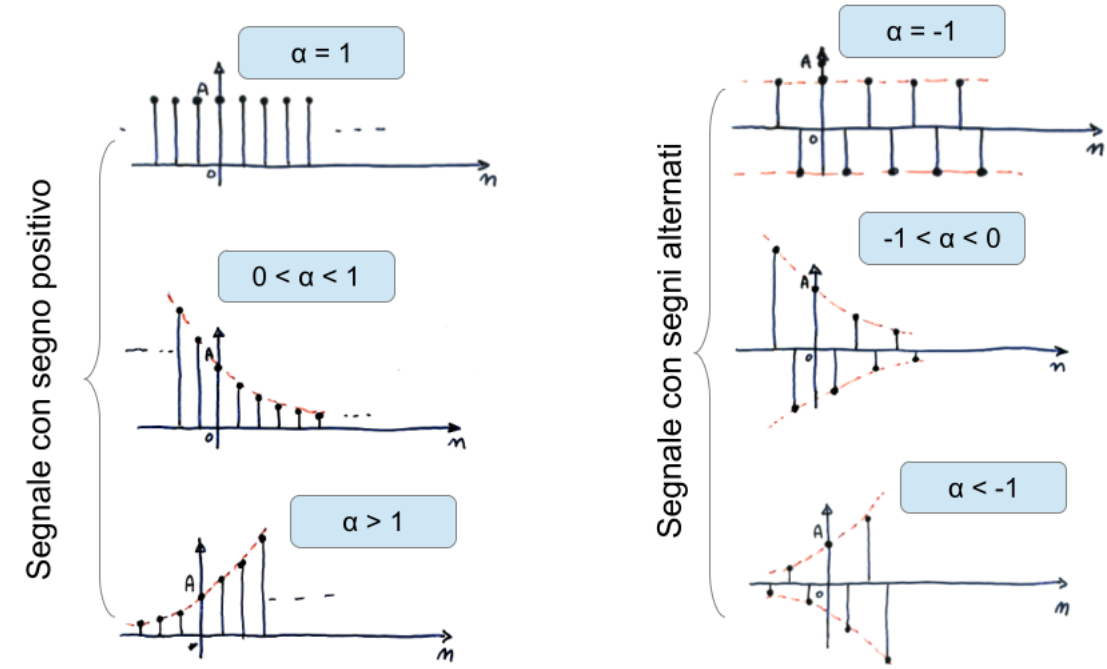
[7]

Di seguito la definizione di sequenza esponenziale reale.

Una sequenza $x[n]$ esponenziale reale ha la seguente formulazione matematica:

$$x[n] = A \cdot \alpha^n \text{ con } A, \alpha \in \mathbb{R} \tag{1.14}$$

In base al valore di α, il segnale avrà i seguenti andamenti:



[7] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 6

1.1.2.4 Sequenze esponenziali complesse

[8]

Di seguito la definizione di sequenza esponenziale complessa.

Una sequenza $x[n]$ esponenziale complessa ha la seguente formulazione matematica:

$$x[n] = A \cdot \alpha^n \text{ con } A, \alpha \in \mathbb{C} \tag{1.15}$$

L'inviluppo della sequenza ha lo stesso andamento dell'esponenziale reale.

> Per inviluppo si intende "l'integrale" della sequenza.
>
> Avendo a che fare con una sequenza sinusoidale (perchè si considerano A e α complessi), allora avremo una sequenza sinusoidale, il cui andamento tende alle figure del sottocapitolo delle "Sequenze esponenziali reali".
>
> Inoltre, essendo una sequenza complessa, e quindi sinusoidale, dovrà essere studiata sia per il modulo che per la fase.

In base ai valori di α, avremo l'andamento l'andamento di $x[n]$:

- $|\alpha| < 1$, allora $x[n]$ è decrescente
- $|\alpha| = 1$, allora $x[n]$ è costante
- $|\alpha| > 1$, allora $x[n]$ è crescente

Possiamo definire i seguenti casi particolari delle sequenza esponenziali complesse.

Definiamo l'esponenziale complesso puro $x[n]$ come:

$$x[n] = |A| \cdot e^{\jmath(\omega n + \phi)} \tag{1.16}$$

Definiamo il coseno reale $x[n]$, grazie alla formula di Eulero, come:

$$\begin{aligned} x[n] &= \cos(\omega n + \phi) \\ &= \frac{e^{\jmath(\omega n+\phi)} + e^{-\jmath(\omega n+\phi)}}{2} \end{aligned} \tag{1.17}$$

Definiamo il seno reale $x[n]$, grazie alla formula di Eulero, come:

$$\begin{aligned} x[n] &= \sin(\omega n + \phi) \\ &= \frac{e^{\jmath(\omega n+\phi)} - e^{-\jmath(\omega n+\phi)}}{2\jmath} \end{aligned} \tag{1.18}$$

[8] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 7
Appunti — 2025-09-24 — pag 4
Latex — 2025-09-30 Mattino — pag 15

1.1.3 Sequenze periodiche

[9]

Per sequenze periodiche si intendono delle sequenze $x[n]$ che hanno la seguente proprietà:

$$x[n] = x[n+N] \tag{1.19}$$

dove N è un numero intero che indica il periodo.

Un tipo di sequenza periodica è l'esponenziale complesso puro:

$$e^{\jmath(\omega n+\phi)} \tag{1.20}$$

in cui:

- ω è la pulsazione, la cui relazione con il mondo fisico si misura in $\frac{rad}{s}$
- $f = \frac{\omega}{2\pi}$ è la frequenza, la cui relazione con il mondo fisico si misura in Hz
- ϕ è la fase iniziale nel periodo

Essendo nel mondo del DSP, quindi delle sequenze e non dei segnali, le sequenze stesse non hanno una grandezza fisica.

Qui sono solo riportate le relazioni dei coefficienti astratti con il mondo reale.

[9] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 8
Appunti — 2025-09-24 — pag 4
Latex — 2025-09-30 Mattino — pag 16

1.1.3.1 Proprietà dell'esponenziale complesso puro

[10]

L'esponenziale complesso puro $e^{\jmath(\omega n+\phi)}$ gode delle seguenti proprietà.

1° **proprietà:**

Una sequenza con pulsazione ω_0 ed una con $\omega_0 + 2\pi$ sono identiche perchè:

$$A \cdot e^{\jmath(\omega_0 n+2\pi n\phi)} = A \cdot e^{\jmath(\omega_0 n+\phi)} \cdot e^{\jmath 2\pi n} \tag{1.21}$$

dove lo spettro è periodico con periodo 2π.

Si considerano equivalentemente:

$$0 \leq \omega \leq 2\pi \text{ oppure } -\pi \leq \omega \leq +\pi \tag{1.22}$$

$$0 \leq f \leq 1 \text{ oppure } -0.5 \leq f \leq +0.5 \tag{1.23}$$

perchè, grazie alla simmetricità dovuta alla periodicità, si può studiare mezzo periodo della sequenza.

2° **proprietà:**

Le "alte" frequenze sono intorno a $\pm\pi$, le "basse" frequenze intorno allo $[\pm\pi, \pm 2\pi]$.

TO-DO: Fare il disegno delle alte e basse frequenze

Essendo la funzione periodica si può scrivere la sequenza proprietà:

$$e^{\jmath[(2\pi-\omega_0)n+\phi]} = e^{\jmath 2\pi n} \cdot e^{\jmath(-\omega_0 n+\phi)} \tag{1.24}$$

3° **proprietà:**

Questa è la proprietà più importante dell'esponenziale complesso !!!

NON TUTTI gli esponenziali complessi sono periodici.

Deve esistere un N per cui:

$$e^{\jmath[\omega_0(n+N)+\phi]} = e^{\jmath(\omega_0 n+\phi)} \tag{1.25}$$

dove:

$$\omega_0 N = 2\pi k \tag{1.26}$$

Se N esiste, allora la sequenza esponenziale complesso puro sarà periodico nel periodo N.

In altre parole, se vogliamo una sequenza esponenziale complessa periodica, i valori di N e di k dipendono tutti da ω_0 e devono rispettare l'uguaglianza scritta qui sopra

[10] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 8 - 9
Appunti — 2025-09-24 — pag 4
Latex — 2025-09-30 Mattino — pag 17

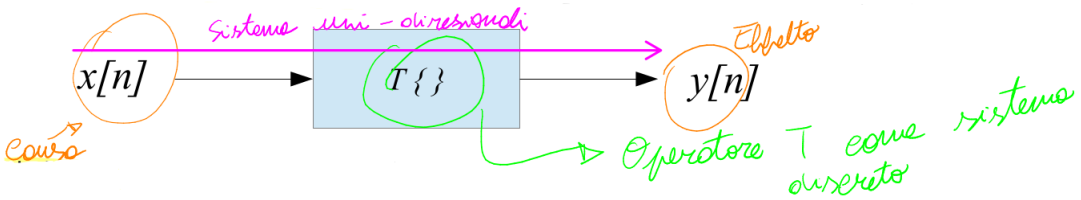
1.2 Sistemi discreti

[11]

Dalle sequenze discrete passiamo ora ai sistemi discreti: di seguito la definizione di sistemi discreti.

Si dice sistema discreto un qualunque operatore (algoritmo o circuito) che trasforma la sequenza di ingesso $x[n]$ in quella di uscita $y[n]$.

Di seguito uno schema di un sistema discreto (con delle note):

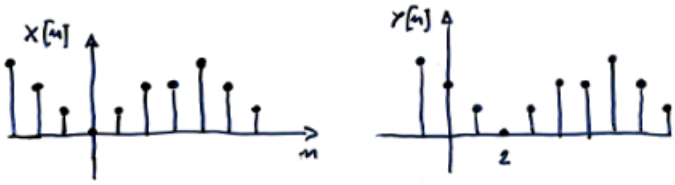


Di seguito degli esempi di sistemi discreti in cui si confrontano l'ingresso $x[n]$ e l'uscita $y[n]$ rispetto al sistema discreto con un solo un ingresso e una uscita.

Un esempio di sistema discreto che implementa un ritardo ideale (che cioè trasla la sequenza nel tempo):

$$y[n] = x[n-2] \tag{1.27}$$

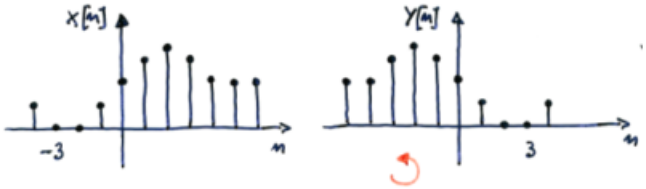
Da un punto di vista grafico:



Un esempio di sistema discreto che implementa un reverse:

$$y[n] = x[-n] \tag{1.28}$$

Da un punto di vista grafico:



Un esempio di sistema discreto che implementa un accumulatore:

$$y[n] = \sum_{k=-\infty}^{n} x[k] \tag{1.29}$$

In particolare, questo ultimo è un sistema con memoria perchè la somma è con i valori precedenti.

[11] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 10
Appunti — 2025-09-24 — pag 4

1.2.1 Sistemi lineari e stazionari

[12]

Un tipo di sistemi molto interessanti ai nostri studi (perchè sono i più semplici da studiare) sono i sistemi discreti lineari e stazionari.

Di seguito una spiegazione più approfondita della proprietà.

[12] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 11
Appunti — 2025-09-30 — pag 1

1.2.1.1 Sistema discreto lineare

[13]

Un operatore (o in questo caso sistema) $T\{\}$ è lineare **se e solo se** vale il principio di sovrapposizione degli effetti.

In formule:

$$\begin{cases} y_1[n] = T\{x_1[n]\} \\ y_2[n] = T\{x_2[n]\} \end{cases} \rightarrow a \cdot y_1[n] + b \cdot y_2[n] = T\{a \cdot y_1[n] + b \cdot y_2[n]\} \quad (1.30)$$

dove a, b sono delle costanti.

In altri termini, se vale il principio di sovrapposizione degli effetti (che è una proprietà dei sistemi lineari), allora possiamo considerare l'uscita del sistema discreto lineare come combinazione lineare degli ingressi.

[13] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 11

1.2.1.2 Sistema discreto stazionario

[14]

Un operatore (o in questo caso sistema) $T\{\}$ è stazionario (cioè a tempo-invariante) **se e solo se** la trasformazione che opera non dipende dall'istante di applicazione.

In formule:

$$\begin{gathered} y[n] = T\{x[n]\} \\ \downarrow \\ y[n-n_0] = T\{x[n-n_0]\} \ \forall n_0, x[n] \end{gathered} \tag{1.31}$$

dove n_0 è un indice di ritardo e $x[n]$ è una sequenza di ingresso all'operatore $T\{\}$.

[14] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 12

1.2.2 Sistemi causali e stabili

[15]

Un tipo di sistemi molto interessanti ai nostri studi (perchè sono i più semplici da studiare) sono i sistemi discreti causali e stabili.

Di seguito una spiegazione più approfondita della proprietà.

La causalità e la stabilità sono proprietà già studiate e approfondite nello scorso corso di elettrotecnica.

Adesso, in questo corso di DSP, affronteremo queste proprietà nel mondo dei sistemi discreti.

[15] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 13

1.2.3 Sistemi discreti causali

[16]

La causalità è una proprietà molto importante dei sistemi discreti, perchè, in base alla sequenza di ingresso, l'andamento del sistema varierà di conseguenza

Un operatore (o in questo caso sistema) $T\{\}$ è causale se la sua uscita all'istante n_0 dipende solo dagli ingressi con indice:

$$n \leq n_0 \tag{1.32}$$

In formule, se:

$$\begin{cases} y_1[n] = T\{x_1[n]\} \\ y_2[n] = T\{x_2[n]\} \end{cases} \tag{1.33}$$

allora:

$$\begin{aligned} x_1[n] &= x_2[n] \\ &\downarrow \\ y_1[n] &= y_2[n] \;\; \forall n_0, x_1[n], x_2[n], n \leq n_0 \end{aligned} \tag{1.34}$$

TO-DO: Fare il disegno dell'andamento di una sequenza dall'indice k

Un esempio di un di sistema causale è la differenza in indietro:

$$y[n] = x[n] - x[n-1] \tag{1.35}$$

Invece, un sistema non causale è la differenza in avanti come il seguente:

$$y[n] = x[n+1] - x[n] \tag{1.36}$$

TO-DO: Fare il disegno dell'andamento di differenza in avanti e una differenza in indietro

[16] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 13
Appunti — 2025-09-30 — pag 1
Latex — 2025-10-03 Pomeriggio — pag 23

1.2.4 Sistemi discreti stabili BIBO

[17]

L'operatore $T\{\}$ si dice stabile BIBO (cioè Bounded Input Bounded Output) se, ad un ingresso **limitato**, corrisponde un'uscita **limitata**.

In formule, se:

$$y[n] = T\{x[n]\} \tag{1.37}$$

allora:

$$\begin{aligned} |x[n]| \leq & B_x < +\infty \\ & \downarrow \\ |y[n]| \leq & B_y < +\infty \ \forall n, x[n] \end{aligned} \tag{1.38}$$

dove B_x e B_y sono dei limiti superiori rispettivamente dell'ingresso e dell'uscita, i quali non tendono ad infinito.

> Per adesso, l'opposto della stabilità, cioè l'instabilità del sistema, non è un problema.
>
> Verrà approfondito in seguito, perchè non è detto che sia un problema: come tutte le cose in ingegneria, dipende.

[17] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 14

1.3 Risposta impulsiva in un sistema LTI

[18]

Per sistema LTI (cioè Linear Time-Invariant) si intende un sistema lineare a tempo invariante, cioè un sistema lineare che non varia le sue caratteristiche nel tempo.

Di seguito la definizione di risposta impulsiva.

Se $T\{\}$ è un operatore LTI, allora è completamente caratterizzato dalla sua risposta all'**impulso unitario**.

In formule, sia:

$$h[n] = T\{\delta[n]\} \tag{1.39}$$

dove $\delta[n]$ è l'impulso unitario, allora possiamo scrivere che:

$$\begin{aligned} y[n] &= T\{x[n]\} \\ &= T\left\{\sum_{k=-\infty}^{+\infty} x[k] \cdot \delta[n-k]\right\} \end{aligned} \tag{1.40}$$

Per linearità, posso portare l'operatore $T\{\}$ dentro alla sommatoria, quindi:

$$\begin{aligned} y[n] &= T\left\{\sum_{k=-\infty}^{+\infty} x[k] \cdot \delta[n-k]\right\} \\ &= \sum_{k=-\infty}^{+\infty} x[k] \cdot T\{\delta[n-k]\} \\ &= \sum_{k=-\infty}^{+\infty} x[k] \cdot h[n-k] \end{aligned} \tag{1.41}$$

> Possiamo svolgere questa semplificazione con $T\{\}$ anche se non sappiamo nel dettaglio come è fatto l'operatore: l'unica cosa che sappiamo è la relazione tra $T\{\}$ e l'ingresso $x[n]$, che in questo caso è $\delta[n]$, cioè l'impulso unitario.

Quindi, possiamo esprimere l'uscita $y[n]$ del sistema LTI, dato come ingresso un impulso unitario, con questa formula:

$$\begin{aligned} y[n] &= \sum_{k=-\infty}^{+\infty} x[k] \cdot h[n-k] \\ &= \sum_{k=-\infty}^{+\infty} x[n-k] \cdot h[k] \end{aligned} \tag{1.42}$$

In questo caso, l'uscita $y[n]$ prende il nome di prodotto di convoluzione.

[18] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 15

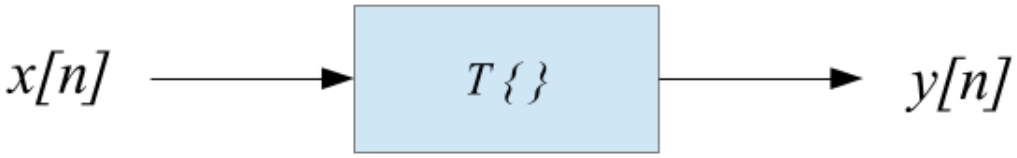
Nel caso "analogico", la sommatoria sarebbe l'integrale di convoluzione, che nel corso di Teoria dei segnali con il mitico Chiaraluce, sarebbe:

$$\begin{aligned} y(t) &= \int_{-\infty}^{+\infty} f(\tau) \cdot g(t-\tau) d\tau \\ &= \int_{-\infty}^{+\infty} f(t-\tau) \cdot g(t) d\tau \end{aligned} \tag{1.43}$$

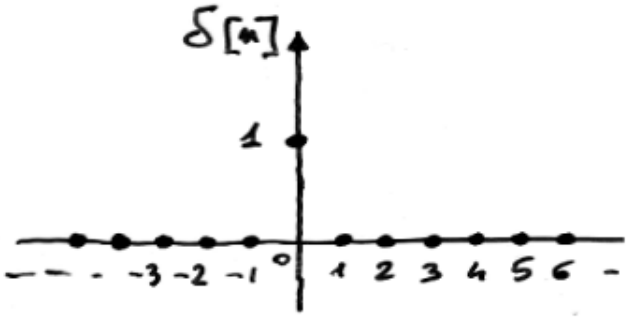
Per approfondire nel mondo analogico:
https://it.wikipedia.org/wiki/Convoluzione

Il prodotto di convoluzione significa che, sapendo che il sistema è LTI, cioè è indipendente dal tempo, basta solo sapere la risposta impulsiva $h[n]$ per conoscere tutto il sistema $T\{\}$.

Graficamente parlando, dato un sistema LTI:



e dato questo tipo di ingresso, cioè l'impulso unitario $\delta[n]$:



che poi deve essere ritardato per ogni indice di k, è possibile ricavare la risposta impulsiva del sistema LTI.

1.3.1 Prodotto di convoluzione e le sue proprietà

[19]

Il seguente prodotto di convoluzione:

$$\begin{aligned} y[n] &= \sum_{k=-\infty}^{+\infty} x[k] \cdot h[n-k] \\ &= \sum_{k=-\infty}^{+\infty} x[n-k] \cdot h[k] \end{aligned} \tag{1.44}$$

può essere anche scritto con la seguente notazione:

$$\begin{aligned} y[n] &= x[n] * h[n] \\ &= h[n] * x[n] \end{aligned} \tag{1.45}$$

Di seguito le proprietà del prodotto di convoluzione.

Proprietà commutativa

Data una sequenza di ingresso $x[n]$ e una risposta impulsiva $h[n]$ di un sistema discreto LTI, allora possiamo applicare la proprietà commutativa nel prodotto di convoluzione, cioè:

$$h[n] * x[n] = x[n] * h[n] \tag{1.46}$$

Proprietà distributiva

Date due sequenze di ingresso $x_1[n]$, $x_2[n]$ e una risposta impulsiva $h[n]$ di un sistema discreto LTI, allora possiamo applicare la proprietà distributiva nel prodotto di convoluzione, cioè:

$$h[n] * (x_1[n] + x_2[n]) = h[n] * x_1[n] + h[n] * x_2[n] \tag{1.47}$$

Proprietà di linearità

Date due sequenze di ingresso $x_1[n]$, $x_2[n]$ con i rispettivi coefficienti moltiplicativi a, b e una risposta impulsiva $h[n]$ di un sistema discreto LTI, allora possiamo applicare la proprietà di linearità nel prodotto di convoluzione, cioè:

$$h[n] * (a \cdot x_1[n] + b \cdot x_2[n]) = a \cdot (h[n] * x_1[n]) + b \cdot (h[n] * x_2[n]) \tag{1.48}$$

[19] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 15 - 16

1.3.2 Come calcolare la risposta impulsiva in un sistema LTI

[20]

La risposta di un sistema può essere calcolata in due metodi.

Il primo metodo è attraverso la sovrapposizione delle risposte del sistema a ciascun impulso $x[k]$, ciascuno con posizione k, cioè:

$$y[n] = \sum_{k=-\infty}^{+\infty} x[k] \cdot T\{\delta[n-k]\} \tag{1.49}$$

dove $T\{\delta[n-k]\}$ è la risposta del sistema discreto LTI all'impulso unitario ritardato dell'indice k.

> Il problema più grosso di questa equazione è la sommatoria con indice k che va da $-\infty$ a $+\infty$, cioè infinite sommatorie.
>
> Inoltre, per lo stesso problema, si dovrebbero fare infiniti ritardi del gradino unitario $\delta[n]$.

Il secondo metodo è quello della somma dei prodotti fra i campioni $x[n]$ e dei campioni della sequenza $h[k]$ rovesciata e spostata di n, cioè (facendo i passaggi step-by-step):

$$\begin{aligned} y[n] &= \sum_{k=-\infty}^{+\infty} x[k] \cdot h[n-k] \\ &= \sum_{k=-\infty}^{+\infty} x[k] \cdot h[-k+n] \\ &= \sum_{k=-\infty}^{+\infty} x[k] \cdot h[-(k-n)] \end{aligned} \tag{1.50}$$

> Rispetto all'equazione precedente, questa volta il ritardo della risposta impulsiva $h[-(k)]$ viene fatta per l'indice n e non per l'indice k.
>
> Inoltre, il ribaltamento (o rovesciamento) di $h[k-n]$ viene fatta grazie al segno negativo davanti a (k - n).
>
> A lezione, per spiegare in modo grafico cosa è la convoluzione, il prof ci ha fatto vedere questi due video:
>
> - https://it.wikipedia.org/wiki/Convoluzione#/media/File:Convolution_of_box_signal_with_itself2.gif
> - https://it.wikipedia.org/wiki/Convoluzione#/media/File:Convolution_of_spiky_function_with_box2.gif

[20] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 17
Appunti — 2025-09-30 — pag 1
Latex — 2025-10-03 Pomeriggio — pag 28

1.3.3 Risposta impulsiva in un sistema causale e stabile BIBO

[21]

Se il sistema è causale, cioè ha la seguente risposta impulsiva:

$$h[n] = 0 \ \forall n < 0 \tag{1.51}$$

allora, la risposta impulsiva al sistema, cioè $y[n]$, diventa:

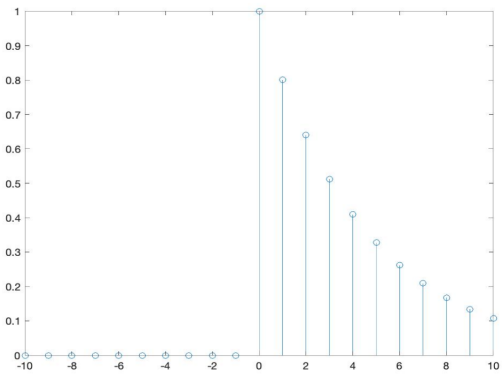
$$\begin{aligned} y[n] &= \sum_{k=-\infty}^{+\infty} x[k] \cdot T\{\delta[n-k]\} \\ &= \sum_{k=-\infty}^{+\infty} x[k] \cdot h[n-k] \\ &= \sum_{k=-\infty}^{+\infty} h[k] \cdot x[n-k] \\ &\downarrow \\ y[n] &= \sum_{k=0}^{+\infty} h[k] \cdot x[n-k] \end{aligned} \tag{1.52}$$

Tra il secondo ed il terzo passaggio si è svolta questa uguaglianza:

$$\sum_{k=-\infty}^{+\infty} x[k] \cdot h[n-k] = \sum_{k=-\infty}^{+\infty} h[k] \cdot x[n-k] \tag{1.53}$$

grazie alla proprietà commutativa del prodotto di convoluzione.

Da un punto di vista grafico, possiamo visualizzare, come esempio, una risposta di un sistema causale $y[n]$ come:



. . .

Come possiamo notare dal grafico, per $k < 0$, la sequenza causale è nulla.

[21] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 18
Appunti — 2025-09-30 — pag 2
Latex — 2025-10-03 Pomeriggio — pag 29

Inoltre, sapendo che nei sistemi causali:

$$h[n] = 0 \ \forall n < 0 \tag{1.54}$$

possiamo far diventare un sistema non causale a causale se aspettiamo sufficientemente indici in modo che si possa fare la convoluzione dall'indice $k = 0$.

Invece, per sistemi stabili BIBO (cioè Bounded Input Bounded Output), in cui:

$$\begin{cases} |x[n]| \leq B_x < +\infty \\ |y[n]| \leq B_y < +\infty \end{cases} \quad \forall n, x[n] \tag{1.55}$$

allora, la risposta impulsiva in un sistema BIBO vale:

$$|y[n]| \leq \sum_{k=-\infty}^{+\infty} |h[k]| \cdot |x[n-k]| \leq B_x \cdot \sum_{k=-\infty}^{+\infty} |h[k]| \tag{1.56}$$

Affinchè ci sia stabilità per qualsiasi sequenza di ingresso $x[n]$, basta che:

$$\sum_{k=-\infty}^{+\infty} |h[k]| < +\infty \tag{1.57}$$

cioè, in altri termini, diremmo che la risposta impulsiva converge; questo ci permette di affermare che la stabilità del sistema LTI la si può osservare anche dalla sua risposta impulsiva.

Grazie alle osservazioni svolte per i sistemi causali ed i sistemi stabili BIBO, possiamo dire che la risposta impulsiva $h[k]$ (o $h[n]$ dipende sempre dall'indice di riferimento), caratterizza completamente un sistema LTI discreto.

1.4 Suddivisione dei sistemi LTI

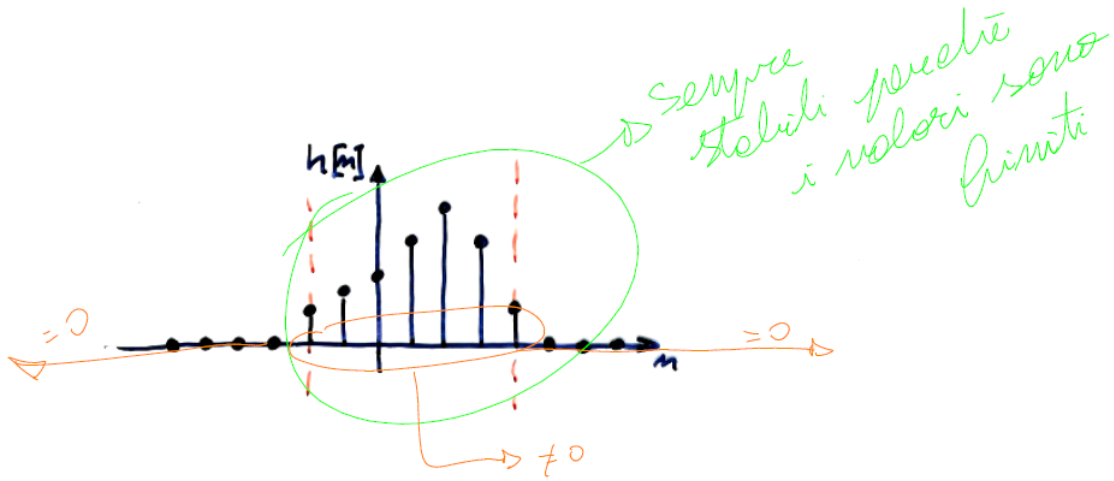
[22]

I sistemi LTI si possono dividere in due tipi:

- FIR (cioè Finite Impulse Response)
- IIR (cioè Infinite Impulse Response)

I sistemi FIR, come dice il nome, sono sistemi in cui la risposta impulsiva $h[n]$ ha durata limitata e che quindi non si estende da $-\infty$ a $+\infty$.

Se consideriamo la finestra compresa tra gli indici n_1 e n_2, possiamo visualizzare un esempio di risposta impulsiva $h[n]$ FIR (con le note prese a lezione):



Possiamo affermare che, un sistema LTI è FIR se:

$$h[n] \neq 0 \text{ solo per } n_1 \leq n \leq n_2 \tag{1.58}$$

Il sistemi LTI FIR saranno sempre stabili se:

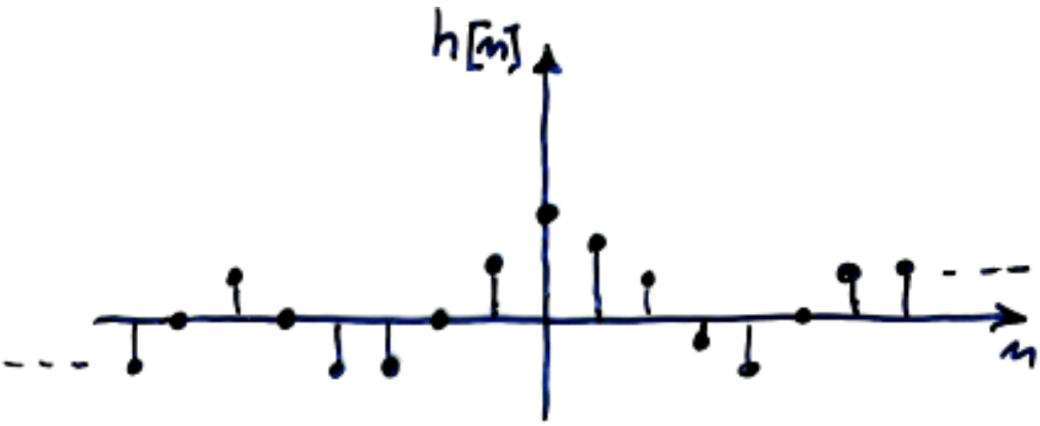
$$|h[n]| < +\infty \ \forall n \tag{1.59}$$

[22] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 19 - 20

Invece i sistemi IIR sono sistemi in cui la risposta impulsiva ha durata illimitata (e quest'ultimo dettaglio può essere un pro ed un contro: lo vedremo meglio nei prossimi capitoli).

Quindi, rispetto al caso precedente dei FIR, la finestra non sarà limitata compresa tra due indici n_1 e n_2, bensì la finestra sarà tra $-\infty$ e $+\infty$.

Un esempio di risposta impulsiva IIR:



dove i puntini a destra e a sinistra della figura indicano che $h[n]$ si estende da $-\infty$ a $+\infty$.

La stabilità del sistema LTI IIR sarà garantita solo se:

$$\sum_{k=-\infty}^{+\infty} |h[k]| < +\infty \tag{1.60}$$

Quindi, per garantire la stabilità del sistema LTI non bisogna che la sua risposta impulsiva debba avere una finestra finita, l'importante che la sua risposta impulsiva converga ad un valore che non sia infinito.

1.4.1 Esempio numerico di stabilità di un sistema IIR

[23]

Ad esempio, data la formulazione matematica di una risposta impulsiva:

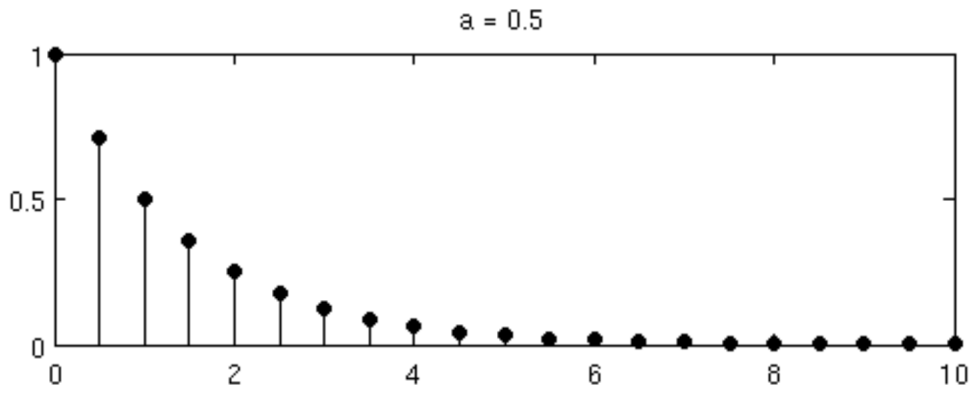
$$h[n] = a^n \cdot u[n] \tag{1.61}$$

dove $u[n]$ è un impulso unitario e a è un coefficiente, possiamo affermare che, se:

$$\begin{cases} h[n] = a^n \cdot u[n] \\ |a| < 1 \end{cases} \tag{1.62}$$

allora il sistema IIR è stabile perchè la risposta impulsiva converge.

Un esempio grafico di $h[n]$ con $a = 0.5$:

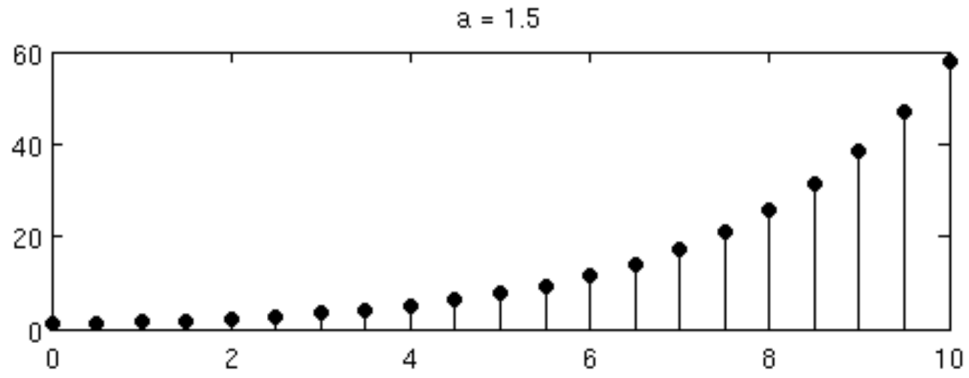


Invece se:

$$\begin{cases} h[n] = a^n \cdot u[n] \\ |a| \geq 1 \end{cases} \tag{1.63}$$

allora il sistema IIR è instabile perchè la risposta impulsiva non converge e tende a infinito.

Un esempio grafico di $h[n]$ con $a = 1.5$:



[23] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 21

1.4.2 Analisi di esempi di sistemi LTI

[24]

Di seguito un elenco di sistemi discreti LTI, come si distinguono per la loro risposta impulsiva $h[n]$e se sono FIR o IIR.

Sistema identità

$$\begin{gathered} y[n] = x[n] \\ \downarrow \\ h[n] = \delta[n] \end{gathered} \tag{1.64}$$

quindi, il sistema identità è un sistema FIR stabile, causale e stabile.

Ritardo di M campioni

$$\begin{gathered} y[n] = x[n-M] \\ \downarrow \\ h[n] = \delta[n-M] \end{gathered} \tag{1.65}$$

Il sistema è FIR e stabile, ma la causalità dipende dai valori di M.

Se $M > 0$, il sistema è causale.

Se $M < 0$, il sistema è non causala.

Differenza in avanti

$$\begin{gathered} y[n] = x[n+1] - x[n] \\ \downarrow \\ h[n] = \delta[n+1] - \delta[n] \end{gathered} \tag{1.66}$$

In questo caso, il sistema è FIR, non causale e stabile.

Differenza all'indietro

$$\begin{gathered} y[n] = x[n] - x[n-1] \\ \downarrow \\ h[n] = \delta[n] - \delta[n-1] \end{gathered} \tag{1.67}$$

In questo caso, il sistema è FIR, causale e stabile.

[24] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 22 - 23
Appunti — 2025-09-30 — pag 2

Accumulatore

$$\begin{aligned} y[n] &= \sum_{k=-\infty}^{n} x[k] \\ &\downarrow \\ h[n] &= \sum_{k=-\infty}^{n} \delta[k] = u[n] \end{aligned} \tag{1.68}$$

dove $u[n]$ è un gradino.

Il sistema è IIR, causale ma non stabile.

Accumulatore ritardato

$$\begin{aligned} y[n] &= \sum_{k=-\infty}^{n} x[k-M] \\ &\downarrow \\ h[n] &= \sum_{k=-\infty}^{n} \delta[k-M] = u[n-M] \end{aligned} \tag{1.69}$$

dove $u[n]$ è un gradino e M è un indice di ritardo.

Il sistema è IIR, non stabile, ma la causalità dipende dal ritardo M.

Se $M > 0$, il sistema è causale (cioè è in ritardo).

Se $M < 0$, il sistema è non causala (cioè è in anticipo).

1.5 Connessioni dei sistemi LTI

[25]

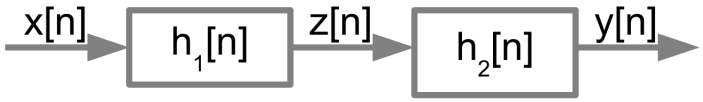
Come abbiamo visto ad elettrotecnica, anche in DSP possiamo connettere i "blocchetti".

Nel caso del DSP, non ci sono rami che sono bidirezionali, bensì unidirezionali.

Di seguito l'elenco delle connessioni di sistemi discreti LTI.

Cascata / Serie

Dati due sistemi discreti LTI, possiamo collegarli in cascata (o in serie) come in figura:



in cui:

- $h_1[n]$ e $h_2[n]$ sono le risposte impulsive dei due sistemi discreti LTI
- $x[n]$ è la sequenza di ingresso
- $z[n]$ è la risposta in uscita al primo sistema LTI
- $y[n]$ è la risposta in uscita da tutti e due i sistemi LTI

Allora possiamo scrivere le seguenti relazioni:

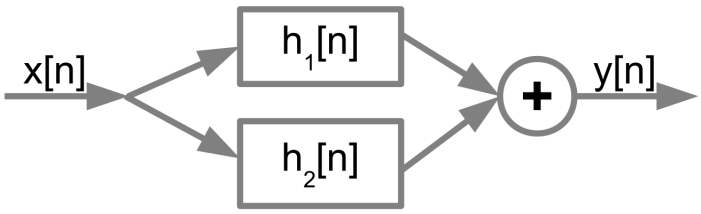
$$\begin{cases} z[n] = h_1[n] * x[n] \\ y[n] = h_2[n] * z[n] \end{cases} \rightarrow y[n] = (h_1[n] * h_2[n]) * x[n] \tag{1.70}$$

L'espressione della sequenza in uscita $y[n]$ esprime che $y[n]$ è combinazione lineare delle risposte impulsive $h_1[n]$ e $h_2[n]$: per questo motivo, volendo, possiamo spostare l'ordine dei due sistemi e l'uscita sarebbe la stessa.

Questo "scambio" non si può svolgere in sistemi discreti non lineari, in cui l'ordine dei blocchi è fondamentale.

Parallelo

Dati due sistemi discreti LTI, possiamo collegarli in parallelo come in figura:



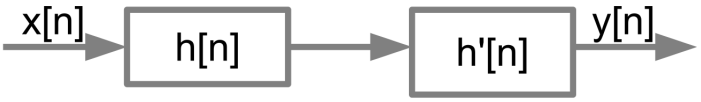
Scrivendo le relazioni matematiche:

$$\begin{aligned} y[n] &= x[n] * h_1[n] + x[n] * h_2[n] \\ &= (h_1[n] + h_2[n]) * x[n] \end{aligned} \tag{1.71}$$

[25] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 24

Sistema inverso

Dato un sistema LTI con una risposta impulsiva $h[n]$ e mettendo in cascata il suo inverso $h^{'}[n]$ come in figura:



risulta che:

$$\begin{aligned} y[n] &= \left(h[n] * h^{'}[n]\right) * x[n] \\ &= x[n] \end{aligned} \tag{1.72}$$

ovvero che:

$$\begin{gathered} h[n] * h^{'}[n] = \delta[n] \\ \downarrow \\ h^{'}[n] \text{ è il sistema inverso di } h[n] \end{gathered} \tag{1.73}$$

Questo tipo di modellazione è molto utile, ad esempio, nei sistemi di telecomunicazione, dove si vuole modellare il canale con disturbo con una risposta impulsiva $h[n]$.

Il ricevitore calcola l'inverso di $h[n]$, in modo da ricevere il messaggio originale inviato dal mittente, cioè:

$$y[n] \approx x[n] \tag{1.74}$$

Ho messo il segno $\approx$ perchè, dal corso di telecomunicazioni, sappiamo bene che è difficile quantificare a priori le caratteristiche dei disturbi in un canale, quindi il massimo che si può fare è dare una rappresentazione semplificata e lineare di esso.

1.6 Sistemi LTI descritti da equazioni alle differenze

[26]

Questi tipi di sistemi vanno memorizzati a memoria !!!

Una sottoclasse di sistemi LTI di importanza pratica, può essere descritta da equazioni alle differenze, cioè:

$$\sum_{k=0}^{N} a_k \cdot y[n-k] = \sum_{k=0}^{M} b_k \cdot x[n-k] \tag{1.75}$$

con a_k e b_k costanti reali.

In altre parole, si può dire che i sistemi LTI descritti da equazioni alle differenze sono quei sistemi in cui la combinazione lineare in uscita è uguale alla combinazione lineare in ingresso.

Tre osservazioni in merito a questo tipo di sistema:

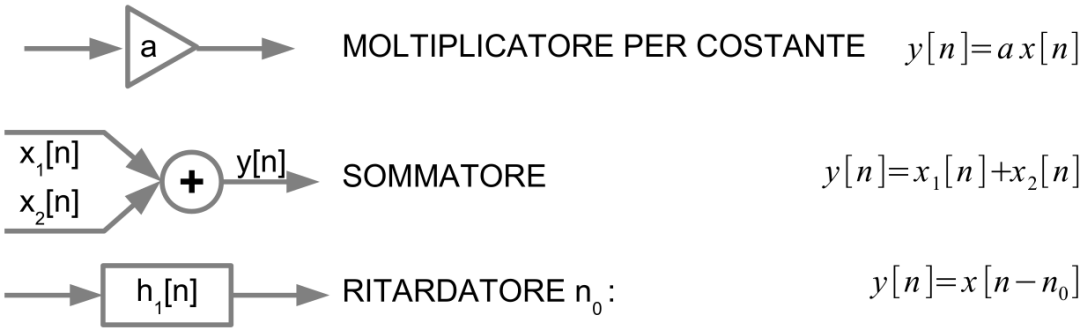
- le sommme sono finite e non infinite come nel caso del prodotto di convoluzione, perchè gli indici vanno da $k = 0$ a N (per l'uscita) ed a M (per l'ingresso)
- non è necessario conoscere la risposta impulsiva del sistema
- è facilmente computabile da un calcolatore

Un caso particolare di sistema discreto LTI descritto da equazioni alle differenze è il seguente:

$$y\,[n] = \sum_{k=0}^{M} b_k \cdot x\,[n-k] \tag{1.76}$$

in cui la risposta impulsiva è finita e causale e la condizione sufficiente affinchè il sistema sia FIR è che $N = 0$.

Tutte le equazioni alle differenze finite, lineari ed a coefficienti costanti possono essere realizzate a partire dai seguenti componenti di base:



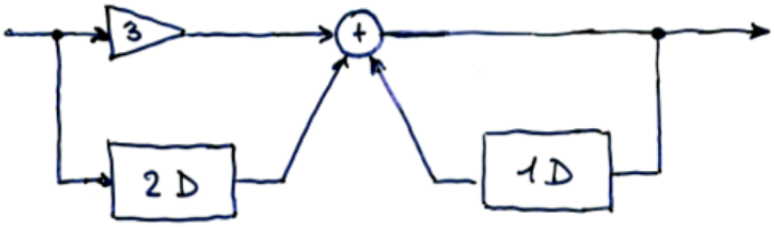
Ovunque c'è ritardo, c'è memoria: dipende se la memoria è finita o infinita (cioè FIR e IIR).

[26] Slide del prof — 1 - Segnali e Sistemi a Tempo Discreto — pag 25 - 26
Appunti — 2025-09-30 — pag 2 - 3

Un esempio di sistema LTI descritto da equazioni alle differenze è la seguente:

$$y[n] - y[n-1] = 3 \cdot x[n] + x[n-2] \tag{1.77}$$

che, utilizzando i componenti di base precedentemente elencati, possiamo rappresentare in questa maniera:



dove con D, nella figura, si intende il ritardo n_0.

Capitolo 2

Introduzione al MATLAB

2.1 Matlab ed i suoi utilizzi

[1]

Il corso di DSP non è un corso di Matlab, ma useremo il programma MATLAB per imparare dei concetti relativi al DSP.

Essendo studenti della mitica UNIVPM e se non avete mai scaricato MATLAB, potete seguire la seguente guida https://www.ict.univpm.it/node/236.

Visto che ci possono essere alcune differenze nell'uso dei comandi (cioè lo stesso comando tra una versione e l'altra può cambiare lievemente funzione), in questo anno accademico 2025/2026, il professore sta utilizzando la versione di MATLAB 2022b, invece nell'aula CESMI, cioè l'aula in cui si svolgerà l'esame, è installata la versione 2023b.

Il mio consiglio è quello di installare tutte e due le versioni con i due toolbox che serviranno all'esame, che sono:

- Signal Processing Toolbox, link della relativa documentazione https://it.mathworks.com/help/signal/index.html
- DSP System Toolbox, link della relativa documentazione https://it.mathworks.com/help/dsp/index.html

Vi consiglio di scaricare ADESSO le versioni di Matlab che vi servono perchè a Maggio 2025 c'è stato un attacco informatico che ha disattivato tutti i servizi di Mathworks per un mese intero https://it.mathworks.com/company/trust-center/may-2025-ransomware-incident.html

Non voglio che, nel mentre che vi state preparando per l'esame, stiate con l'ansia che questo attacco possa capitare di nuovo e non potervi esercitare con questo programma.

Se non avete mai programmato in vita vostra (non siamo nati tutti imparati, sennò non saremmo andati all'università), la documentazione è quel file (ma generalmente è un sito) in cui potete approfondire meglio quello specifico argomento.

Inoltre, Matlab mette a disposizione anche dei video come il seguente sia nel proprio sito web https://it.mathworks.com/videos/introduction-to-signal-processing-apps-in-matlab-1580188288390.html, che nel suo canale su Youtube https://www.youtube.com/@MATLAB/featured.

Se siete alle prime armi, o anche per un semplice ripasso in compagnia, vi consiglio il seguente video (in cui utilizza Matlab 2020b ma le basi, come scritto precedentemente, più o meno sono le stesse):

MATLAB Crash Course for Beginners by freeCodeCamp.org
https://www.youtube.com/watch?v=7f50sQYjNRA

Negli appunti cercherò anche, se possibile, di spiegare il codice passo passo (anche perchè all'esame ci verrà richiesto di scrivere uno script Matlab).

Ora basta con le doverose introduzioni, proseguiamo.

Matlab è un ambiente di sviluppo per algoritmi orientato all'elaborazione di matrici.

È un linguaggio ad "alte" prestazioni per problemi computazionali.

Matlab è disponibile su diverse piattaforme, MacOS, Linux, Windows (e anche sul cloud) e, ogni anno, vengono rilasciate due versioni: nella prima metà dell'anno esce la versione a e nella seconda metà dell'anno esce la versione b.

[1] Slide del prof — 2 - Introduzione al MATLAB — pag 2 - 3
Appunti — 2025-09-30 — pag 3 - 5

Generalmente è consigliato il Matlab per una prototipazione di un progetto e non da utilizzarlo per un progetto finale perchè è un linguaggio molto lento, cioè per svolgere una istruzione ci mette tanto tempo e tante risorse (consiglio il Matlab per persone che devono sostituire il termosifone con il proprio pc in cameretta d'inverno): generalmente vengono utilizzati, successivamente, altri linguaggi più efficienti per finalizzare il progetto, come ad esempio il C/C++.

Oltre al Matlab si possono installare dei Toolbox, che sono dei programmi accessori al Matlab (un po' come delle espansioni nei giochi da tavola, ma in questo caso per Matlab).

Di seguito i Toolbox utili per il corso di DSP:

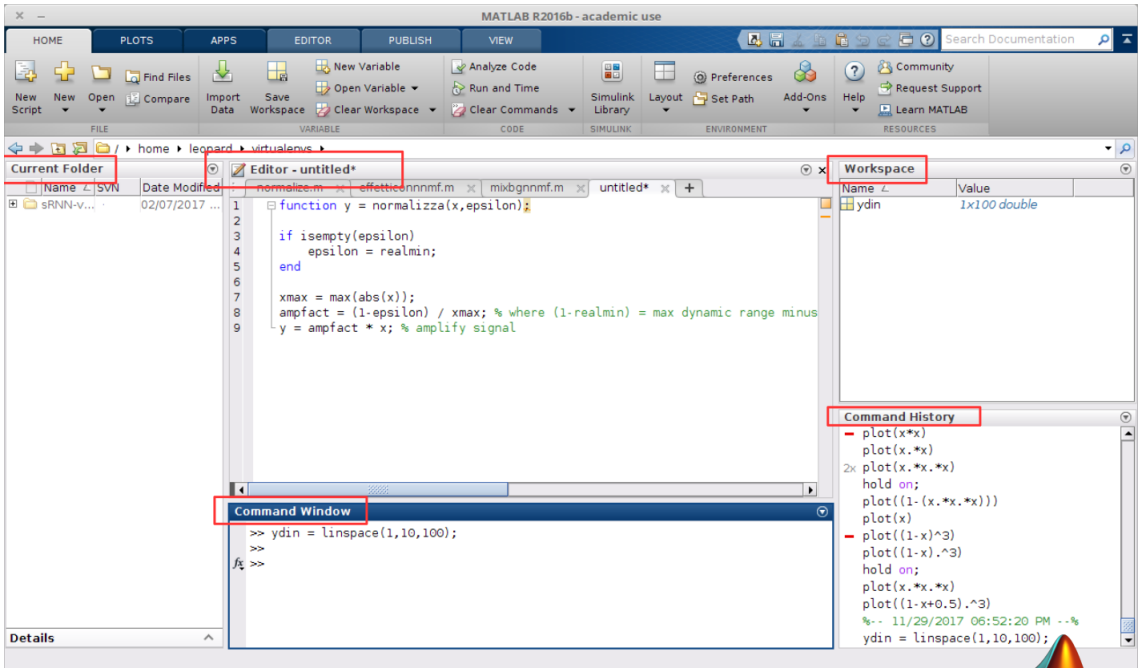
- Signal Processing Toolbox https://it.mathworks.com/products/signal.html
- DSP System Toolbox https://it.mathworks.com/products/dsp-system.html

2.2 Il sistema Matlab, l'IDE ed i Data Types

[2]

Il sistema Matlab offre diverse possibilità di funzioni, ma noi ci concentreremo sulle principali.

Di seguito uno screenshot all'interno di Matlab, con delle note:



La command history è la finestra in cui potrete visualizzare i comandi più recenti.

Il workspace è la finestra in cui vengono memorizzati i dati più recenti.

La set path è quella dialog box dove indica la directory (cioè la cartella) in cui si trova il Matlab.

La command window è quella finestra dell'IDE per eseguire i comandi di Matlab.

Di seguito alcuni comandi più utilizzati, con gli usi più frequenti e con il link alla documentazione:

- save, per salvare un elemento da Matlab su un file esterno
  https://it.mathworks.com/help/matlab/ref/save.html
- load, per caricare un file esterno come elemento su Matlab
  https://it.mathworks.com/help/matlab/ref/load.html
- help, per visualizzare come può essere utilizzato un comando
  https://it.mathworks.com/help/matlab/ref/help.html
- clear, per cancellare delle variabili dal workspace
  https://it.mathworks.com/help/matlab/ref/clear.html
- whos, per avere delle informazioni dettagliate su un file
  https://it.mathworks.com/help/matlab/ref/whos.html
- pwd, per indicare la cartella in cui ci si trova
  https://it.mathworks.com/help/matlab/ref/pwd.html
- cd, per cambiare cartella
  https://it.mathworks.com/help/matlab/ref/cd.html
- dir, per elencare i file presenti nella cartella
  https://it.mathworks.com/help/matlab/ref/dir.html

[2] Slide del prof — 2 - Introduzione al MATLAB — pag 6 - 9

Scrivo che generalmente quel tipo di comando viene utilizzato in questa maniera perchè, se leggete la documentazione, lo stesso comando, in base alla versione ed ai parametri che inserite, comporta ad esiti differenti.

Durante l'esame scritto non potrete andare su internet per cercare di scrivere il proprio script: ecco perchè il comando help e la documentazione interna saranno i vostri alleati.

Se possibile, imparate già da adesso a leggerla ed a capirla, in modo che all'esame non perdiate del tempo: sarà solo un veloce ripasso leggere la documentazione.

In Matlab i dati possono essere di diversi tipi.

Di seguito elenco i principali:

- Array multidimensionali classici, cioè scalari, vettori, matrici (come quelli che hai studiato nel corso di Algebra Lineare)
- Array di celle, cioè array multidimensionali i cui elementi sono copie di altri array (un po' come le matrioske russe)
- Caratteri, cioè degli array ottenuti scrivendo il testo tra apici
- Strutture, cioè array multidimensionali accessibili attraverso designatori di campo

Per array si intende, dall'inglese, vettore.

In questo corso, verranno utilizzati principalmente gli array multidimensionali e gli array di celle.

Altro particolare di Matlab è che è un "linguaggio di programmazione" case-sensitive, cioè un linguaggio in cui le lettere maiuscole e minuscole distinguono due tipi di elementi.

Ad esempio: pippo e Pippo sono due variabili in Matlab differenti.

Inoltre, nel Matlab, gli indici partono da 1 e non da 0, come ad esempio nel C

2.3 Operazioni su matrici e vettori

[3]

Matlab è un linguaggio di programmazione orientato ai vettori e alle matrici, quindi vediamo un po' di codici di esempi per generarli.

> Se volete approfondire le operazioni che si possono fare con le matrici su Matlab https://it.mathworks.com/help/matlab/math/matrices-in-the-matlab-environment.html

Generazione di matrici

Di seguito una generazione di una matrice M nella command window:

```
>> M=[1 0 -1; 0 2 1; 0 0 3]

M =

     1     0    -1
     0     2     1
     0     0     3
```

> In base a come abbiamo scritto il comando, la variabile M è una matrice 3x3, cioè 3 righe e 3 colonne.
>
> Per generare il vettore M bisogna inserire i valori all'interno delle parentesi quadre, cioè [].
>
> Il punto e virgola (;) permette di scrivere la riga successiva.

Estrazione di una sottomatrice

Di seguito l'estrazione di una sotto-matrice da command window in cui è disponibile la matrice M nel workspace:

```
>> M(1:2,2:3)

ans =

     0    -1
     2     1
```

> Con quel comando indicato nella command window, si intende che si vuole prendere la sottomatrice di M che va dalla prima riga alla seconda riga e dalla seconda colonna alla terza colonna.
>
> In questo caso, si utilizzano i due punti (:) per dire da quale riga/colonna si vuole partire a sinistra dei due punti, a quella riga/colonna indicato a destra dei due punti.

[3] Slide del prof — 2 - Introduzione al MATLAB — pag 10 - 13
Appunti — 2025-10-01 — pag 1 - 3

Operatore colon ":"

Di seguito un esempio dell'operatore colon ":" usato frequentemente nei comandi:

```
>> a=[0:0.1:1]

a =

  Columns 1 through 7

         0    0.1000    0.2000    0.3000
0.4000    0.5000    0.6000

  Columns 8 through 11

    0.7000    0.8000    0.9000    1.0000
```

L'operatore colon ":" è molto utile perchè piuttosto che scrivere:

```
a = [0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1]
```

Il comando:

```
a = [0: 0.1 : 1]
```

è molto più rapido e agevole.

In altre parole il comando esprime di creare un vettore che parte dal valore iniziale 0 fino al valore finale 1 a passo di 0.1.

Il passo può essere anche negativo.

Funzioni matriciali

Se volete approfondire le operazioni che si possono fare con le matrici su Matlab
https://it.mathworks.com/help/matlab/math/matrices-in-the-matlab-environment.html

Esistono molte funzioni matriciali già implementate in Matlab.

Ad esempio, il comando:

```
M_trasposta = M'
```

implementa la funzione matriciale:

$$M_{trasposta} = M^T \tag{2.1}$$

dove M è una matrice e con T si intende la sua trasposta.

Se non ti ricordi cosa significa fare la matrice trasposta:

Matrice Trasposta by Wikipedia
https://it.wikipedia.org/wiki/Matrice_trasposta

Il comando sum in esempio:

```
vect = [1 0 -1 2 1 4 2];
somma_elementi_vettore = sum(vect)
```

somma tutti gli elementi contenuti nel vettore vect, cioè implementa la seguente funzione matematica:

$$vect = 1 + 0 - 1 + 2 + 1 + 4 + 2 \tag{2.2}$$

Il comando diag in esempio:

```
diag(M)
```

calcola elenca gli elementi di una diagonale di una matrice.

> Per apprfondire il comando diag: https://it.mathworks.com/help/matlab/ref/diag.html
>
> Se non ti ricordi cosa significa la diagonale di una matrice:
>
> Matrice diagonale by Wikipedia
> https://it.wikipedia.org/wiki/Matrice_diagonale

Il comando eig in esempio:

```
eig(M)
```

calcola gli autovalori della matrice M.

> Per approfondire il comando eig: https://it.mathworks.com/help/matlab/ref/eig.html
>
> Se non ti ricordi cosa significa la diagonale di una matrice:
>
> Eigenvectors and eigenvalues — Chapter 14, Essence of linear algebra by 3Blue1Brown
> https://youtu.be/PFDu9oVAE-g?si=wJ0_hB40hehyk4Qq

Il comando per il vettore M2:

```
b = [5 5 5];
M2 = [M; b]
```

permettono di concatenare due o più matrici (in questo caso la matrice M2 è concatenata alla matrice M e alla matrice b).

> Per visualizzare un esempio di concatenazione di matrici:
>
> Matrix Concatenation in MATLAB by thesalmonification
> https://youtu.be/TAIetkMXmLs?si=Qf_-3bJuiA75J65_

Oltre alle funzioni matriciali, è possibile definire delle matrici multidimensionali, i quali vengono trattate come strutture bidimensionali.

Ad esempio i seguenti comandi:

```
A = randn(2, 3, 2);
A(:, :, 1)
```

permettono di creare una matrice multidimensionale con dimensione 2x3x2 con valori casuali.

> Per approfondire gli array multidimensionali: https://it.mathworks.com/help/matlab/math/multidimensional-arrays.html

Oltre a questo, è possibile svolgere delle operazioni matriciali come le seguenti:

```
+ (addition)
- (subtraction)
* (multiplication)
/ (division)
\ (left division, see Matrices and
Linear Algebra in Matlab Doc)
```

Inoltre, utilizzando il punto prima dell'operatore, si può effettuare una operazione elemento per elemento (cioè element-wise) tra i vettori e matrici coinvolti.

Un approfondimento su questo argomento: https://it.mathworks.com/help/matlab/matlab_prog/array-vs-matrix-operations.html

Di seguito un esempio di comandi utilizzati comunemente su Matlab:

```
 Elementary matrices.
   zeros        - Zeros array.
   ones         - Ones array.
   eye          - Identity matrix.
   repmat       - Replicate and tile array.
   repelem      - Replicate elements of an array.
   linspace     - Linearly spaced vector.
   logspace     - Logarithmically spaced vector.
   freqspace    - Frequency spacing for frequency response.
   meshgrid     - X and Y arrays for 3-D plots.
   accumarray   - Construct an array with accumulation.
   :            - Regularly spaced vector and index into matrix.

 Basic array information.
   size         - Size of array.
   length       - Length of vector.
   ndims        - Number of dimensions.
   numel        - Number of elements.
   disp         - Display matrix or text.
   isempty      - True for empty array.
   isequal      - True if arrays are numerically equal.
   isequaln     - True if arrays are numerically equal, treating NaNs
as equal.

 Matrix manipulation.
   cat          - Concatenate arrays.
   reshape      - Reshape array.
   diag         - Diagonal matrices and diagonals of matrix.
   blkdiag      - Block diagonal concatenation.
   tril         - Extract lower triangular part.
   triu         - Extract upper triangular part.
   fliplr       - Flip matrix in left/right direction.
   flipud       - Flip matrix in up/down direction.
   flip         - Flip the order of elements.
   rot90        - Rotate matrix 90 degrees.
   :            - Regularly spaced vector and index into matrix.
   find         - Find indices of nonzero elements.
   end          - Last index.
   sub2ind      - Linear index from multiple subscripts.
   ind2sub      - Multiple subscripts from linear index.
   bsxfun       - Binary singleton expansion function.
```

```
 Multi-dimensional array functions.
   ndgrid       - Generate arrays for N-D functions and interpolation.
   permute      - Permute array dimensions.
   ipermute     - Inverse permute array dimensions.
   shiftdim     - Shift dimensions.
   circshift    - Shift array circularly.
   squeeze      - Remove singleton dimensions.

 Array utility functions.
   isscalar     - True for scalar.
   isvector     - True for vector.
   isrow        - True for row vector.
   iscolumn     - True for column vector.
   ismatrix     - True for matrix.

 Special variables and constants.
   eps          - Floating point relative accuracy.
   realmax      - Largest positive floating point number.
   realmin      - Smallest positive floating point number.
   intmax       - Largest positive integer value.
   intmin       - Smallest integer value.
   flintmax     - Largest consecutive integer in floating point
format.
   pi           - 3.1415926535897....
   i            - Imaginary unit.
   inf          - Infinity.
   nan          - Not-a-Number.
   isnan        - True for Not-a-Number.
   isinf        - True for infinite elements.
   isfinite     - True for finite elements.
   j            - Imaginary unit.
   true         - True array.
   false        - False array.

 Specialized matrices.
   compan       - Companion matrix.
   gallery      - Test matrices.
   hadamard     - Hadamard matrix.
   hankel       - Hankel matrix.
   hilb         - Hilbert matrix.
   invhilb      - Inverse Hilbert matrix.
   magic        - Magic square.
   pascal       - Pascal matrix.
   rosser       - Classic symmetric eigenvalue test problem.
   toeplitz     - Toeplitz matrix.
   vander       - Vandermonde matrix.
   wilkinson    - Wilkinson's eigenvalue test matrix.
```

Come scritto nei capitoli precedenti, il tempo nel DSP non esiste perchè siamo nell'ambito dei vettori.

Generalmente assoceremo il tempo in base a dove si trova nel vettore il valore (cioè l'indice del vettore sarà per noi l'istante temporale).

I numeri complessi esistono in Matlab: la parte immaginaria viene indicata o con la variabile i o con la variabile j.

Ad esempio se abbiamo il numero:

$$\overrightarrow{A} = 3 + 3\jmath \tag{2.3}$$

in Matlab lo possiamo scrivere come:

```
A = 3 + 3i
```

oppure:

```
A = 3 + 3j
```

TO-DO: Spiegazione codice Esempio 2.1

2.4 Plotting

[4]

In Matlab, ci sono diversi modi di graficare i vettori i vettori, che sono:

- plot, in cui vengono interpolati (cioè collegati) i punti in "modo" continuo https://it.mathworks.com/help/matlab/ref/plot.html
- stem, in cui viene data una rappresentazione ad impulsi https://it.mathworks.com/help/matlab/ref/stem.html
- bar, in cui viene data una rappresentazione con i diagrammi a barre https://it.mathworks.com/help/matlab/ref/bar.html

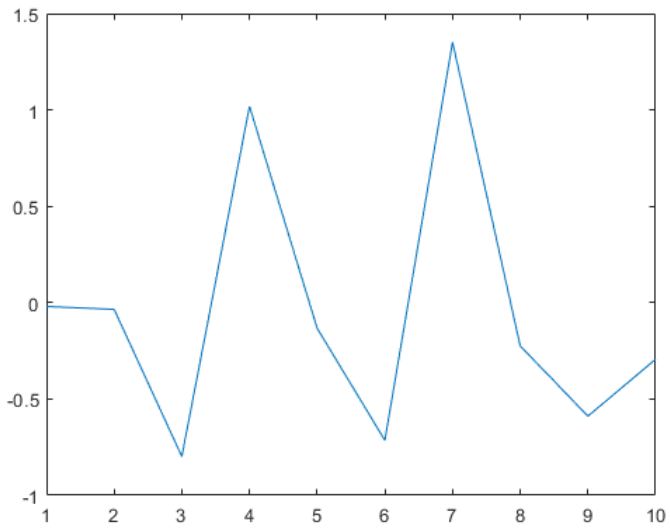
> Se vuoi sapere tutti i modi per graficare un vettore: https://it.mathworks.com/help/matlab/creating_plots/types-of-matlab-plots.html

Se si invoca un solo vettore, esso verrà graficato rispetto all'indice.

Ad esempio dato il vettore con valori casuali A:

```
A = randn(1, 10);
plot(A)
```

avremo il seguente caso:



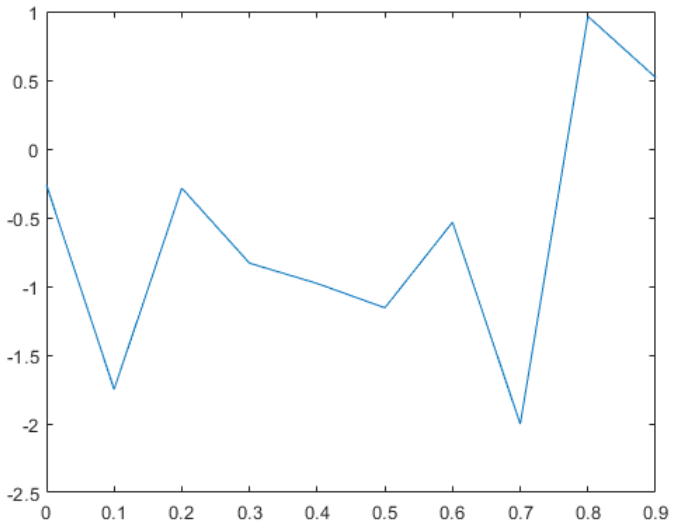
dove, nell'asse delle x o chiamato anche l'asse delle ascisse, viene utilizzato l'indice del vettore.

[4] Slide del prof — 2 - Introduzione al MATLAB — pag 14 - 15

Se invece si invocano due vettori, il primo è utilizzato nell'asse delle x, cioè nell'asse delle ascisse, ed il secondo per l'asse y, cioè quello delle ordinate.

Ad esempio, dati i seguenti vettori A e B in cui B ha valori casuali:

```
A = [0: 0.1 : 1]
B = randn(1, 10);
plot(A, B)
```



Quindi, in questo caso ad esempio, il vettore A può essere utilizzato per indicare "il tempo", invece il vettore B per indicare i valori istantanei della velocità di un maratoneta.

Di default, Matlab dirige l'output dei comandi grafici sulla finestra figure più recente (ad esempio su Windows si apre un'altra finestra oltre a quella dell'IDE di Matlab chiamata, ad esempio, Figure 1).

I seguenti comandi sono utili per gestire più figure in Matlab:

- figure, crea una nuova finestra "figure" e vi dirige l'output
- figure(N), dirige l'output sulla finestra N
- hold on, abilita la sovrapposizione dei grafici sull'ultima figure attivata
- hold off, disabilita la sovrapposizione
- sublot(r, c, i), suddivide la finestra in una matrice di sottografici di r righe, c colonne e direzione l'output sul sottografico i-esimo (contando per righe)

Per approfondire i comandi figure, hold, subplot:

- figure https://it.mathworks.com/help/matlab/ref/figure.html
- hold https://it.mathworks.com/help/matlab/ref/hold.html
- subplot https://it.mathworks.com/help/matlab/ref/subplot.html

TO-DO: Spiegazione codice Esempio 2.2

2.5 Esercizi di esempio

[5]

TO-DO: Spiegazione codice Esempio 2.1, 2.2, 2.3, 2.4

[5] Slide del prof — 2 - Introduzione al MATLAB — pag 16 - 17
Appunti — 2025-10-01 — pag 3

2.6 Matlab Programming

[6]

È possibile realizzare programmi ad-hoc tramite il linguaggio di programmazione Matlab.

Il linguaggio Matlab, oltre alle semplificazioni assegnazioni di variabili e ad operatori matematici, comprende anche "statements" più evoluti come:

- if, elseif, else, eseguono delle operazione se le condizioni sono vere https://it.mathworks.com/help/matlab/ref/if.html
- switch, case, otherwise, eseguono uno tra i gruppi di operazioni https://it.mathworks.com/help/matlab/ref/switch.html
- for, foor-loop che si ripete un numero di volte prefissato https://it.mathworks.com/help/matlab/ref/for.html
- while, while-loop che si ripete se la condizione rimane vera https://it.mathworks.com/help/matlab/ref/while.html
- break, termina un'esecuzione di un while-loop o di un for-loop https://it.mathworks.com/help/matlab/ref/break.html

Esistono due possibilità per programmare in Matlab:

- script
- funzioni

[6] Slide del prof — 2 - Introduzione al MATLAB — pag 18
Appunti mancanti — 2025-10-07

2.6.1 Script file

[7]

Il file script è un file che contiene una sequenza di comandi Matlab, generalmente con estensione .m.

È bene non assegnare a nessun script il nome delle funzioni predefinite o il nome di una variabile usata.

TO-DO: Spiegazione codice Esempio 2.5

[7] Slide del prof — 2 - Introduzione al MATLAB — pag 19 - 20
Appunti mancanti — 2025-10-07

2.6.2 Funzioni

[8]

Una funzione si crea come uno script: ha estensione .m, deve avere lo stesso nome della funzione, ma, a differenza dello script, può restituire dei risultati i cui parametri avvengono per valore.

Il passaggio per valore implica che le variabili definite all'interno della funziona hanno visibilità locale e non possono essere riferite dalla shell.

> I concetti di visibilità locale e globale sono dei concetti molto importanti nella programmazione.
>
> Per approfondire: https://it.wikipedia.org/wiki/Visibilit%C3%A0_(programmazione)
>
> Per quanto riguarda il passaggio del valore in una funzione, lo si ha anche in altri linguaggi di programmazione, come ad esempio il C e il C++.
>
> Per approfondire la differenza tra passaggio per valore e passaggio per riferimento:
>
> - Passaggio per valore
> https://www.ibm.com/docs/it/i/7.5.0?topic=calls-pass-by-value
> - Passaggio per riferimento
> https://www.ibm.com/docs/it/i/7.5.0?topic=calls-pass-by-reference

In Matlab esiste un tipo di variabile interessante chiamata come "Persistent variable", che sono variabili locale che sono viste come globali solamente dalla funzione dove sono state definite.

> Nella documentazione di persistent sono disponibili degli esempi più dettagliati per approfondire il tema: https://it.mathworks.com/help/matlab/ref/persistent.html

[8] Slide del prof — 2 - Introduzione al MATLAB — pag 21

2.7 Convoluzione di due sequenze

[9]

Come studiato dal capitolo precedente, sappiamo bene che la convoluzione tra la sequenza discreta di ingresso ad un sistema discreto LTI $x[n]$ e la risposta impulsiva $h[n]$, è definita come:

$$y[n] = \sum_{k=-\infty}^{+\infty} x[k] \cdot h[n-k] \tag{2.4}$$

In Matlab esiste una funzione pre-implementata per il calcolo della convoluzione tra due sequenze ed è la seguente:

```
y = conv(x, h)
```

Ma, come si legge bene dalla documentazione di conv https://it.mathworks.com/help/matlab/ref/conv.html, essa produce un vettore y di lunghezza pari a:

```
length(x) + length(h) - 1
```

mentre noi vorremmo che la lunghezza del vettore y sia:

```
length(x) + length(h)
```

Per questo motivo che, in questo corso, useremo la funzione my_conv.m .

TO-DO: Spiegazione codice Esempio 2.3 my_conv.m

Come studiato nel capitolo precedente 1.1.1 Proprietà di campionamento una qualsiasi sequenza $x[n]$ può essere espressa come somma di impulsi unitari opportunamente ritardati e amplificati.

Da un punto di vista matematico, possiamo scrivere la sequenza $x[n]$ come:

$$x[n] = \sum_{k=-\infty}^{+\infty} x[k] \cdot \delta[n-k] \tag{2.5}$$

dove la sequenza di impulsi unitari $\delta[n]$ prende il nome di Delta di Kronocker.

È giusto dire che la sequenza di impulsi unitari $\delta[n]$ prende il nome di Delta di Kronocker?

La proprietà di campionamento delle sequenze discrete appena descritta, permette di assegnare uno specifico significato operativo all'operazione di convoluzione.

TO-DO: Spiegazione codice Esempio 2.4

Tale significato operativo può essere analogamente essere messo in luce considerando la convoluzione come una operazione matriciale.

L'operazione di convoluzione tra due sequenze può essere vista come il prodotto matriciale tra una delle due sequenze e la matrice di convoluzione, opportunamente costruita a partire dalla sequenza rimanente.

[9] Slide del prof — 2 - Introduzione al MATLAB — pag 22

Eh?

La matrice di convoluzione $[H]$ è una matrice di Toepliz rettangolare di dimensione N * p dove:

- N è la lunghezza del prodotto di convoluzione
- p è la lunghezza della sequenza x che non genera la matrice

Possiamo dire che una matrice di Toepliz utile per la proprietà di campionamento nei sistemi discreti è ottenuta utilizzando la Delta di Kronocher, che appunto alla fine compone una matrice identità?

Per approfondire cosa è una matrice di Toepliz e la Delta di Kronocher:

- https://it.wikipedia.org/wiki/Matrice_di_Toeplitz
- https://it.wikipedia.org/wiki/Delta_di_Kronecker

La matrice è ottenuta shiftatndo nel tempo la sequenza base h su ogni colonna.

Per esempio:

TO-DO: Fare un esempio di una convoluzione tra due sequenze utilizzando la matrice di Toeplitz

Questa interpretazione è molto utile ai fini computazionali.

Il comando Matlab da utilizzare per generare la matrice di convoluzione è convmtx.

Dalla documentazione di convmtx https://it.mathworks.com/help/signal/ref/convmtx.html

"Computing a convolution using conv when the signals are vectors is generally more efficient than using convmtx. For multichannel signals, convmtx might be more efficient."

convmtx ha lo stesso problema del comando conv in Matlab, cioè della sua grandezza che non rispecchia quella ideale come noi la intendiamo nel corso?

TO-DO: TO-DO: Spiegazione codice Esempio 2.5, 2.6, 2.7